

HANDS-ON DATA EXTRACTION

Web Scraping & API Integration

Pulling Data from E-Commerce Sites

From a single `requests.get()` to a concurrent, ethical crawler — BeautifulSoup, XPath, Selenium & Scrapy, plus the security lab that explains why bot defenses exist.

What we will build

1 — Scraping vs APIs, and the ETL mindset

3 — HTML & the DOM, then BeautifulSoup

5 — Dynamic pages with Selenium & explicit waits

7 — Specialized + no-code extraction tools

2 — : zero-pain environments & drivers

4 — BeautifulSoup vs XPath, easy → hard

6 — Four hard cases, one slide each

8 — Security lab: bot defenses & how to respect them

Two ways to get the data

Call the API

```
import requests
r = requests.get(
    "https://api.site.com/products",
    params={"page": 1},
)
data = r.json()
```

- Clean, structured **JSON**
- Stable & fast — **always prefer this**
- Look in DevTools → **Network** → **XHR**

Scrape the HTML

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html, "lxml")
name = soup.select_one(".product .title").text
```

- Works when there is **no public API**
- Fragile: breaks when markup changes
- Heavier & slower to parse

Rule of thumb: **find the API first**; scrape the DOM only when you must.

The pipeline: Extract → Transform → Load

Extract

```
r = requests.get(url)
r.encoding = "utf-8"
soup = BeautifulSoup(r.text, "lxml")
```

Pull raw HTML / JSON from the source.

Transform

```
from price_parser import Price
price = Price.fromstring("£16.99").amount
rating = stars_to_int(node["class"])
```

Clean, type-cast, normalize.

Load

```
df.to_sql("products", conn,
         if_exists="replace")
```

Persist to SQLite / Postgres / CSV.

Each repo `step` is one stage — run them in order, or run `pipeline.py` end-to-end.

Environments & drivers with

```
# one tool, no venv juggling
uv sync                # core deps
uv sync --extra notebook
uv sync --extra security

uv run python pipeline.py --pages 5
uv run jupyter lab
```

- Fast resolver written in Rust — seconds, not minutes
- Reproducible via 
- **Selenium Manager** auto-downloads the matching browser driver — no manual chromedriver
- Optional extras keep the base install light

No more "works on my machine."  gives every student the same environment.

What is HTML? Tags, attributes & the DOM

A web page is a tree of **nested elements**. The browser parses it into the **DOM** — the tree we actually scrape.

```
<div class="product" data-sku="A12">
  <h3 class="title">Bike Helmet</h3>
  <span class="price">£24.99</span>
  <a href="/p/a12">View details</a>
</div>
```

- **Element** = opening tag + attributes + content + closing tag
- **Attributes** we target: `class`, `id`, `href`, `data-*`
- Nesting builds the **tree** that BeautifulSoup & XPath walk

Careful: the data may not be there yet

- **Static page** — data already lives in the HTML, so `requests` can read it
- **JS app** — the HTML arrives as an **empty shell**; JavaScript fetches & fills it **after load**

Dynamic content: the data isn't in the HTML
The first response is an empty shell — JavaScript fills it in *after load*. `requests` can't see it.

requests.get(url).text

```
<body>
<div id="quotes"><!-- empty --></div>

// nothing here yet...
<script src="/app.js"></script>
</body>

→ soup.select(".quote") == []
```

Browser / Selenium after WebDriverWait

"The world as we have created it is a process of our thinking."
— Albert Einstein
[change](#) [thinking](#)

"It is our choices that show what we truly are."
— J.K. Rowling
[choices](#)

"There are only two ways to live your life."
— Albert Einstein
[life](#) [miracle](#)

We'll crawl these with Selenium + `WebDriverWait` later

Fundamentals: requests + BeautifulSoup

```
import requests
from bs4 import BeautifulSoup

resp = requests.get("https://books.toscrape.com/")
resp.encoding = "utf-8" # avoid £ → Â£ mojibake
soup = BeautifulSoup(resp.text, "lxml")

for card in soup.select("article.product_pod"):
    title = card.h3.a["title"]
    price = card.select_one(".price_color").text
    print(title, price)
```

Set the encoding explicitly, prefer the `lxml` parser, and select with CSS first.

BeautifulSoup vs XPath — same goal, two dialects

Goal	BeautifulSoup (CSS)	XPath (lxml)
By class	<code>select(".price")</code>	<code>//*[@class="price"]</code>
By id	<code>select_one("#main")</code>	<code>//*[@id="main"]</code>
Attribute	<code>select('[data-sku]')</code>	<code>//*[@data-sku]</code>
Contains text	<code>find(string=...)</code>	<code>//*[contains(text(),"Add")]</code>
Nth child	<code>select("li:nth-child(2)")</code>	<code>(//li)[2]</code>
Climb to ancestor	<code>el.find_parent("div")</code>	<code>ancestor::div ... </code>

CSS is concise for common cases; XPath wins when you must climb **up** the tree or match text.

The gotcha: `contains()` is substring-based

```
# BAD - also matches class="product-list-wrapper"
//div[contains(@class, "product")]

# GOOD - token-safe class match
//div[contains(concat(' ', normalize-space(@class), ' '), ' product ')]
```

- `contains(@class, "product")` matches any class containing that substring
- Pad with spaces and `normalize-space()` to match a **whole class token**
- The same trap exists in CSS when you over-broaden selectors

Dynamic pages: Selenium + explicit waits

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By

el = WebDriverWait(driver, 10).until(
    EC.presence_of_element_located(
        (By.CSS_SELECTOR, ".product-card")
    )
)

# NEVER do this in production:
# time.sleep(5)    # guesses, flaky, slow
```

Replace blind `sleep()` with `WebDriverWait` + `expected_conditions` — wait for the **condition**, not the clock.

The four "hard" cases

01 — Complex pagination

02 — Lazy-load / infinite scroll

03 — `WebDriverWait` done right

04 — Data trapped in `<table>`

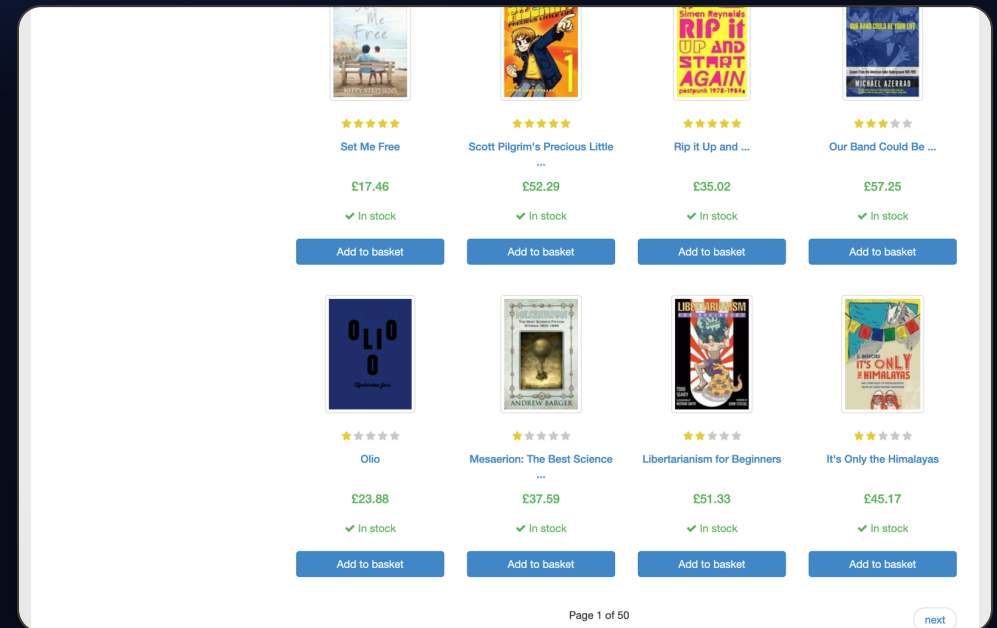
One slide each — what it looks like in the wild, and how to crawl it.

Hard case 1 — Complex pagination

Many listings split results across numbered pages or a "next" link.

- **URL-based** — loop `?page=n` with plain `requests` (cheap & fast)
- **JS "Load more"** — click the button, then `WebDriverWait` for fresh cards
- Stop when the **next** link disappears or a page repeats

```
n = 1
while True:
    r = requests.get(f"{base}?page={n}")
    cards = parse(r.text)
    if not cards:
        break
    n += 1
```



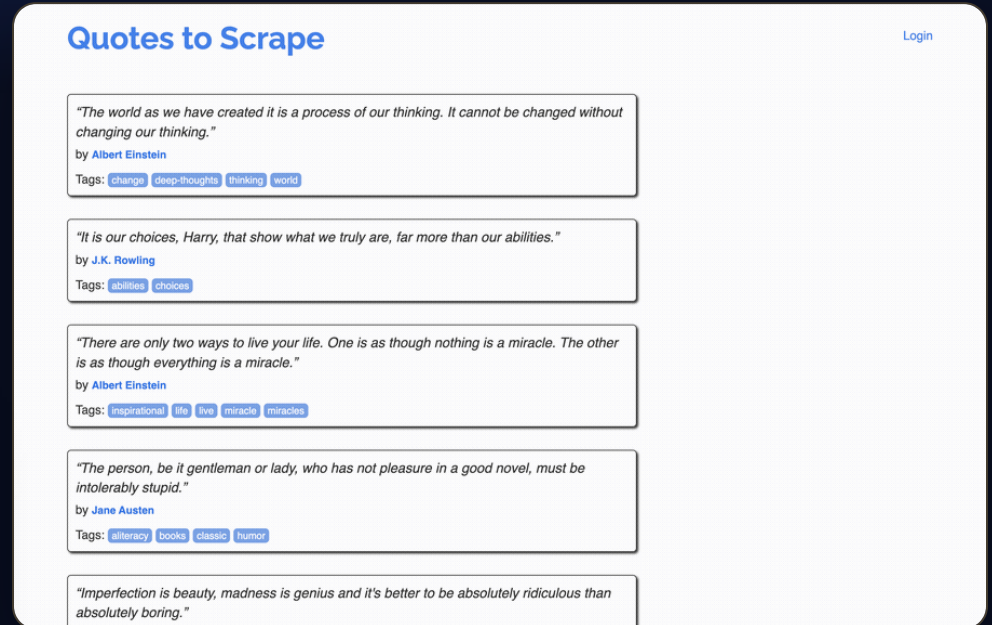
books.toscrape.com — "Page 1 of 50 · next →"

Hard case 2 — Lazy-load / infinite scroll

New items appear only as you scroll — there are no page numbers.

- **Best:** find the hidden **JSON API** the page calls (DevTools → Network → XHR)
- **Fallback:** scroll loop in Selenium until the count stops growing

```
last = 0
while True:
    driver.execute_script(
        "window.scrollTo(0, document.body.scrollHeight)")
    WebDriverWait(driver, 10).until(more_loaded)
    n = len(driver.find_elements(By.CSS_SELECTOR, ".quote"))
    if n == last: break
    last = n
```



quotes.toscrape.com/scroll — 10 → 20 → 30 items as you scroll

Hard case 3 — WebDriverWait done right

The first HTML response is often an **empty shell** — JavaScript fills it in later, so `requests` sees nothing.

- Render in a real browser, then **wait for the element**, not the clock
- `presence_of_element_located` · `visibility_of` · `element_to_be_clickable`
- Never paper over flakiness with `time.sleep()`

```
e1 = WebDriverWait(driver, 10).until(
    EC.presence_of_element_located(
        (By.CSS_SELECTOR, ".quote")))
```

Dynamic content: the data isn't in the HTML
The first response is an empty shell — JavaScript fills it in *after* load. `requests` can't see it.

requests.get(url).text

```
<body>
<div id="quotes"><!-- empty --></div>

// nothing here yet...
<script src="/app.js"></script>
</body>

→ soup.select(".quote") == []
```

Browser / Selenium after WebDriverWait

"The world as we have created it is a process of our thinking."
— Albert Einstein
[change](#) [thinking](#)

"It is our choices that show what we truly are."
— J.K. Rowling
[choices](#)

"There are only two ways to live your life."
— Albert Einstein
[life](#) [miracle](#)

Left: what `requests` sees · Right: what the browser renders

Hard case 4 — Data trapped in `<table>`

Don't hand-walk `<tr>`/`<td>` — let pandas read the whole table at once.

- `pd.read_html` returns a list of DataFrames
- Handle thousands separators & the right header row
- Then clean, type-cast, and load like any other source

```
import pandas as pd
tables = pd.read_html(html, flavor="lxml",
                      thousands=",")
df = tables[0]          # pick the right one
df = df.dropna(how="all")
```

Note: A numbered rank is assigned to the 193 [member states of the United Nations](#), plus the 2 [observer states to the United Nations General Assembly](#). Dependent territories and constituent countries that are parts of sovereign states are not assigned a numbered rank. In addition, [sovereign states with limited recognition](#) are also included, but not assigned a numbered rank.

	Location	Population	% of world	Date	Source (official or from the United Nations)	Notes
	World	8,232,000,000	100%	13 Jun 2025	UN projection ^{[1][3]}	
1	 India	1,417,492,000	17.2%	1 Jul 2025	Official projection ^[4]	^[b]
2	 China	1,404,890,000	17.0%	31 Dec 2025	Official estimate ^[5]	^[c]
3	 United States	341,784,857	4.1%	1 Jul 2025	Official estimate ^[6]	^[d]
4	 Indonesia	288,315,089	3.5%	31 Dec 2025	National annual projection ^[7]	
5	 Pakistan	241,499,431	2.9%	1 Mar 2023	2023 census result ^[8]	^[e]
6	 Nigeria	223,800,000	2.7%	1 Jul 2023	Official projection ^[9]	
7	 Brazil	213,421,037	2.6%	1 Jul 2025	Official estimate ^[10]	
8	 Bangladesh	169,828,911	2.1%	14 Jun 2022	2022 census result ^[11]	^[f]
9	 Russia	146,028,325	1.8%	1 Jan 2025	Official estimate ^[13]	^[g]
10	 Mexico	131,150,866	1.6%	31 Mar 2026	National quarterly estimate ^[14]	
11	 Japan	122,810,000	1.5%	1 May 2026	Monthly national estimate ^[15]	
12	 Philippines	114,123,600	1.4%	1 Jul 2025	Official projection ^[16]	
13	 Democratic Republic of the Congo	112,832,000	1.4%	1 Jul 2025	UN projection ^[17]	

Appearance hide

Text

☐ Small

☒ Standard

☐ Large

Width

☒ Standard

☐ Wide

Color

☐ Automatic

☒ Light

☐ Dark

Wikipedia — countries by population (a classic `<table>`)

NOTEBOOKS/MISC

Specialized extraction tools

newspaper4k — article text

extract — schema.org data

price-parser — money & locale

Stop reinventing parsers — reach for the tool built for the job.

newspaper4k — clean article text in one call

```
from newspaper import Article

a = Article("https://news.site/story")
a.download()
a.parse()

a.title          # headline
a.text           # body, boilerplate stripped
a.authors        # byline
a.publish_date   # parsed datetime
```

- Strips nav, ads & footers — just the **story**
- Works across **many news sources** with the same code
- Also pulls title, authors, top image, publish date
- Great for building a news corpus fast

When you need readable text, not a bespoke selector per site.

extract — read the structured data shops already embed

```
import extract

data = extract.extract(
    html,
    syntaxes=["json-ld", "microdata", "opengraph"],
)
product = data["json-ld"][0]
product["name"], product["offers"]["price"]
```

- Most e-commerce ships **schema.org** JSON-LD for SEO
- Read **name**, **price**, **sku**, **availability** — **no fragile selectors**
- Survives redesigns far better than CSS/XPath
- **Reality check**: not every site provides it → have a fallback

Always check the page source for a `<script type="application/ld+json">` first.

price-parser — currency & locale-aware numbers

```
from price_parser import Price
```

```
Price.fromstring("Rp 1.250.000").amount # 1250000
```

```
Price.fromstring("£16.99").amount # 16.99
```

```
Price.fromstring("$1,299.00").currency # "$"
```

- Handles `.` vs `,` as thousands / decimal across locales
- Separates the **amount** from the **currency** symbol
- Far safer than `re.sub(r"^\d+", "", text)`
- One less source of silent data-quality bugs

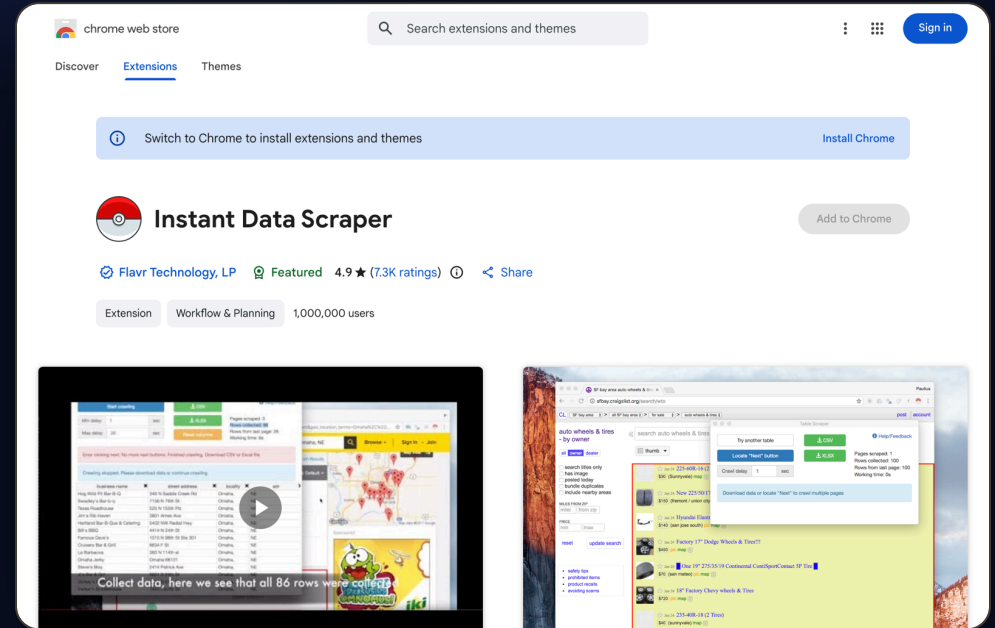
The "Transform" step of ETL, solved for money.

No-code option — Instant Data Scraper (Chrome)

A browser **extension** that auto-detects the main table/list on a page and exports **CSV / XLSX** — no code.

- Great for **quick one-offs** & for non-developers
- It even clicks **"Next"** to follow pagination
- Limits: brittle on heavy JS, hard to schedule, no version control
- Use it to **prototype**, then graduate to `requests` / Scrapy for production

Knowing the no-code path helps you scope when real code is worth it.



Chrome Web Store — point, detect, export to CSV/XLSX

Scrapy: concurrency that scales

```
class ProductSpider(scrapy.Spider):
    name = "products"
    custom_settings = {
        "CONCURRENT_REQUESTS": 16,
        "DOWNLOAD_DELAY": 0.25,
        "AUTOTHROTTLING_ENABLED": True,
        "ROBOTSTXT_OBEY": True,
    }
    async def start(self):
        for p in range(1, 51):
            yield scrapy.Request(url(p))
```

Approach	50 pages
----------	----------

requests (serial)	~25 s
-------------------	-------

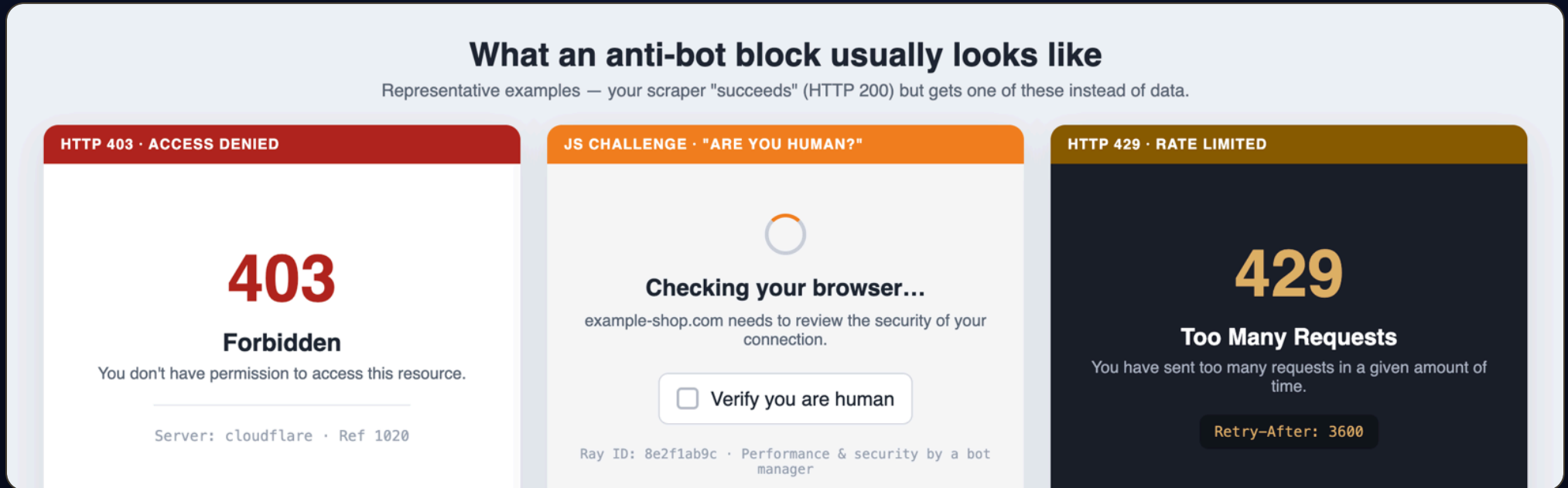
Scrapy (async, x16)	~3 s
---------------------	------

- Async I/O — many requests in flight
- Built-in **AutoThrottle** & retries
- `ROBOTSTXT_OBEY` on by default here
- Pipe items straight into a `DataFrame`

Why bot defenses exist

We study how scrapers bypass protections so we can defend against them. Educational use only — always obey `robots.txt`, Terms of Service, and rate limits.

What a block actually looks like



- A request can return **HTTP 200** yet contain a challenge, not your data
- Watch for tiny HTML, a "verify you are human" box, or a `Retry-After`
- Triggered by default User-Agents, datacenter IPs, no JS, or too-fast rates
- **Blue team takeaway:** these screens are the signal your defenses fired

The escalation ladder

Level	Technique	Typical result vs a WAF
L0	Plain <code>requests</code> , default UA	Blocked / <code>403</code>
L1	Real browser headers + UA	Sometimes through
L2	TLS impersonation (<code>curl_offi</code>)	Defeats JA3/JA4 fingerprinting
L3	Warm-up session → <code>__cf_bm</code> cookie + TLS	Reaches the JSON API

A modern WAF is **stateful** — results hinge on cookies, IP reputation, and session "warmth".

Speak the language — security glossary

- **UA** — User-Agent string identifying the client
- **WAF** — Web Application Firewall / bot manager
- **JA3 / JA4** — fingerprint of the TLS handshake
- `__cf_bm` — Cloudflare bot-management cookie
- **Impersonation** — mimic a real browser's TLS + headers
- **Warm-up** — first request to earn trust / cookies
- **Rate limiting** — caps requests per IP / window
- **Honeytoken** — hidden trap link only bots follow

Blue Team: how to defend

Signal	Defense
Odd TLS fingerprint	JA3/JA4 checks via a WAF / CDN
Robotic request rate	Rate limiting + adaptive throttling
Missing browser cookies	Managed challenge before sensitive routes
Datacenter IPs	IP reputation & ASN scoring
Crawlers ignoring rules	<code>robots.txt</code> + honeypot trap links

Defense in depth: no single check is enough — layer them.

THE ETHICAL LINE

Scrape responsibly

Respect `robots.txt` and Terms of Service · throttle your requests · cache & never re-hammer · identify your bot · prefer official APIs · collect only what you need.

YOU'RE READY

Go start scraping

[walkthrough.ipynb](#)

[beautifulsoup_vs_xpath](#)

[cases/01-04](#)

[misc/scrapy_real_case](#)

github.com/thosangs/dibimbing_scraping · [Project home](#)